技 术 文 档

图书借阅管理系统

项 目 说 明 书

面向对象程序设计（C++）课程设计

文档类型	项目技术说明书
对应程序	Library V1.3（C++17 命令行应用，11 项功能）
代码规模	12 个代码文件，共 2247 行
开发语言	C++17
编译环境	MinGW-w64 g++ 16.2.0 + Visual Studio Code
编写日期	2026 年 10 月 29 日

文档信息..... 1

1 文档说明..... 2

2 项目概览..... 4

3 项目结构..... 6

4 总体设计..... 9

5 各文件详解..... 12

6 关键流程说明..... 36

7 语法与机制专题..... 39

8 数据文件规范..... 43

9 编译与运行配置..... 45

10 维护指南..... 47

11 附录..... 49

结语..... 51

文档信息

项目	内容
文档名称	图书借阅管理系统 项目说明书
文档版本	V1.3
编写日期	2026 年 10 月 29 日
对应程序版本	Library V1.3（12 个代码文件，2247 行，11 项菜单功能）
开发语言	C++17
编译工具	MinGW-w64 g++ 16.2.0
开发环境	Visual Studio Code + C/C++ 扩展
本次修订	① 菜单精简为 11 项并把编号重排为 1~10 连续、`0` 为退出；② 界面标题中的馆藏单位名称去除；③ 新增 `Library::defaultDataDir()`，按可执行文件位置定位数据目录，消除“换启动方式就读到另一份 data”的隐患。全文代码清单、行数统计、流程图与速查表同步更新
适用对象	课程答辩评审、后续维护人员、代码阅读者

1 文档说明

1.1 编写目的

本文档是《图书借阅管理系统》的技术说明书，用于完整交代程序的组织方式与实现细节。具体解决三类问题：

1. **看得懂**：逐一说明每个文件、每个代码区块承担什么职责，读者无需通读源码即可建立整体认识。
2. **讲得清**：对每个代码区块注明所使用的 C++ 语法点，以及该语法在此处解决了什么问题。
3. **改得动**：给出关键流程的调用链与数据流向，说明扩展点位置，便于后续维护与二次开发。

1.2 读者对象

读者	建议阅读路径
课程答辩评审	第 2 章 → 第 4 章 → 第 6 章 → 第 10 章
首次接触代码的维护者	第 3 章 → 第 5 章（按文件顺序通读）
需要修改业务规则者	第 6 章 → 第 9 章 → 第 10 章
需要移植或重构者	第 4 章 → 第 7 章 → 第 8 章

1.3 术语与约定

术语	含义
资源	馆藏对象的统称，包含普通图书、期刊杂志、电子资源三类
条码号	馆内唯一标识，形如 `B001`、`M001`、`E001`，不可修改
书号	国际标准号，包括 ISBN-13、ISBN-10、ISSN-8
借阅人	使用图书馆借阅服务的对象，分为学生与教师
副本	同一种资源的多册馆藏，用“馆藏册数 / 可借册数”表示
基类指针	类型为 `BookResource*` 的指针，可指向

术语	含义
	任意派生类对象

代码引用约定：文中代码块为便于说明做过裁剪，省略号表示被省略的重复代码；完整实现以工程源文件为准。

2 项目概览

2.1 项目背景与目标

本项目依据《面向对象程序设计（C++）实验指导书（2024 版）》题目三实现，目标是把实验一至实验六的知识点整合为一个可运行的命令行应用：用类的封装描述业务实体，用继承与多态描述类型层次，用组合与依赖描述对象协作，用模板容器组织批量对象，用文件流实现数据持久化。

2.2 功能范围

分类	功能	菜单编号
馆藏管理	关键字查询、添加资源、删除资源	1 ~ 3
借阅人管理	添加借阅人、删除借阅人	4 ~ 5
借还业务	办理借书、办理还书、在借记录、逾期清单	6 ~ 9
统计分析	馆藏与借阅统计	10
数据管理	退出时自动保存、启动时自动装载	0

编号在精简后重新排为 1~10 连续，`0` 固定为保存退出，菜单分组（【馆藏】【读者】【借还】【统计】）只作分类提示、不占用编号。被移除的 5 项功能是显示全部馆藏、修改资源、增减馆藏数量、显示全部借阅人与修改借阅人，它们在精简前的菜单中分别位于 1、5、6、7、10。明确不做的部分：不提供多用户登录与权限控制，不提供网络接口，不涉及图形界面，逾期罚款仅计算与提示，不做实际扣款。

2.3 运行环境与技术栈

项目	要求
操作系统	Windows 10/11（程序含 Windows 控制台 API 调用，Linux 下需替换清屏方式）
编译器	支持 C++17 的 g++（工程在 MinGW-w64 g++ 16.2.0 验证）
依赖库	仅 C++ 标准库，无第三方库
关键语言特性	类与继承、纯虚函数、虚析构、类模板、标准容器 `vector`、文件流、

项目	要求
	`std::filesystem`、`std::function`

选用 C++17 的两个直接原因：`std::filesystem` 用于创建数据目录，`std::function` 用于把查找条件传入容器封装。

2.4 设计原则

1. **单一职责**：工具、数据模型、业务逻辑、界面四层分离，界面类不直接改动数据，业务类不直接打印菜单。
2. **面向基类编程**：容器保存基类指针，遍历时调用虚函数，新增资源类型不需改动业务代码。
3. **数据合法性前置**：所有可写入的数据先经校验函数检查，非法输入在入口处被拒绝，避免出现非法状态的对象。
4. **资源所有权明确**：堆对象由 `DataStore` 独占并统一释放，禁止容器拷贝，从设计上排除重复释放。

3 项目结构

3.1 目录结构

```
Library/
├── main.cpp 程序入口，38 行
├── Utils.h / Utils.cpp Text、Date、Console 三个工具类
├── DataStore.h 类模板，仅头文件
├── BookResource.h / .cpp BookResource 基类 + Book、Magazine、EResource
├── Borrower.h / .cpp LoanRecord + Borrower 基类 + Student、Teacher
├── Library.h / .cpp 图书馆整体类，业务规则与文件读写
├── ConsoleUI.h / .cpp 控制台界面，菜单与输入
├── data/ 运行期生成 resources.txt、readers.txt、loans.txt
├── build/ 编译输出 Library.exe
├── .vscode/ tasks.json、launch.json、c_cpp_properties.json
├── build.bat 一键编译脚本
├── README.md 快速上手说明
└── docs/ 本说明书所在目录
```

工程共 12 个代码文件，全部位于同一目录。这样安排的依据是：头文件与源文件同目录时，`#include "BookResource.h"` 由预处理器按“包含者所在目录”直接找到，不需要任何 `-I` 参数，工程搬迁后不会因包含路径失效而无法编译。文件划分遵循“一个业务模块对应一组 .h/.cpp”，同一模块内的类存在强耦合（基类与派生类、整体与部分），合并在同一文件中比拆成多个小文件更利于阅读。

3.2 文件职责矩阵

文件	行数	包含的类	核心职责
`main.cpp`	38	无（仅使用类）	初始化编码、定位数据目录、创建图书馆对象、启动界面、退出保存
`Utils.h` / `Utils.cpp`	81 / 319	`Text`、`Date`、`Console`	字符串切分与转换、日期计算、控制台输入输出与编码设置
`DataStore.h`	72	`DataStore<T>`	以 `vector` 存储对象指针，提供增删查找遍历与统一释放
`BookResource.h`	161 / 358	`BookResource`、	定义馆藏资源的数

文件	行数	包含的类	核心职责
/ .cpp		Book、Magazine、EResource	据结构与三类借阅规则
Borrower.h / .cpp	147 / 304	LoanRecord、Borrower、Student、Teacher	定义借阅人、借阅记录，实现借书与还书的判定逻辑
Library.h / .cpp	60 / 418	Library	组织资源与借阅人集合，执行业务规则，读写数据文件
ConsoleUI.h / .cpp	38 / 251	ConsoleUI	显示菜单、收集输入、调用业务函数、输出结果

3.3 文件之间的依赖关系

```
main.cpp
 ├── Utils.h （使用 Console::init、捕获 Console::ExitSignal）
├─ Library.h ──┬── DataStore.h
│  ├── BookResource.h ─── Utils.h（Text、Date 声明）
│  └─ Borrower.h ──────── Utils.h
└─ ConsoleUI.h ── Library.h
```

依赖方向始终由“上层”指向“下层”：入口依赖业务与界面，界面依赖业务，业务依赖数据模型，数据模型依赖工具。反向依赖被刻意避免——Library 不知道界面存在，BookResource 不知道 Library 存在。唯一处于中间的 Borrower.cpp 通过前向声明 class BookResource; 在头文件中避免直接包含 BookResource.h，把编译依赖留到实现文件，降低了头文件的耦合度。

3.4 分层架构

层次	组成	说明
表示层	ConsoleUI、Console	只负责输入输出，不含业务判断
业务层	Library	组织集合、执行规则、协调借还双方
模型层	BookResource 及其派生类、Borrower 及其派生类、LoanRecord	保存状态，封装与自身相关的行为
基础设施层	Text、Date、DataStore<T>	与业务无关的通用能力

分层的判据是“谁可以被替换而不影响其他层”：界面换成图形界面时只改 `ConsoleUI`；数据换成数据库时只改 `Library` 的读写函数；新增资源类型时只改模型层。

4 总体设计

4.1 类的清单与职责

类	类型	职责	关键成员
Text	工具类（全静态）	字符串切分、去空白、安全数值转换、金额格式化	split、trim、toInt、money
Date	值类型	日期合法性校验与天数差计算	toSerial、daysBetween、addDays
Console	工具类（全静态）	控制台编码设置与带校验的输入	init、readInt、confirm
DataStore<T>	类模板	管理一组堆对象指针，负责释放	add、indexOf、removeAt、forEach
BookResource	抽象基类	馆藏公共属性与统一接口	takeOne、isValidStdNumber、纯虚函数组
Book	派生类	普通图书，借期 30 天，日罚金 0.20 元	pages_、category_
Magazine	派生类	期刊，借期 7 天，日罚金 0.50 元	issue_、cycle_
EResource	派生类	电子资源，借期 3 天，日罚金 0.10 元	url_、sizeMB_
LoanRecord	值类型	一条借阅记录，计算逾期天数	code_、borrowDate_、dueDate_
Borrower	抽象基类	借阅人公共属性与借还行为	loans_、borrowResource、returnResource
Student	派生类	学生，上限 5 册	major_、className_
Teacher	派生类	教师，上限 10 册，罚款减半	title_
Library	整体类	组合资源与借阅	resources_、

类	类型	职责	关键成员
		人，业务规则与持久化	`readers_`、`saveAll`
`ConsoleUI`	界面类	单层菜单与交互流程	`run`、`doBorrow`

4.2 继承体系

程序中有两条互不相关的继承链，分别描述“被借的东西”和“借东西的人”。

```
BookResource（抽象类）
┌──────────────────┼──────────────────┐
Book Magazine EResource
普通图书 30 天 期刊 7 天 电子资源 3 天
0.20 元/天 0.50 元/天 0.10 元/天

Borrower（抽象类）
┌──────────────────┼──────────────────┐
Student Teacher
上限 5 册 上限 10 册
罚款系数 1.0 罚款系数 0.5
```

基类集中共同点，派生类只表达差异：`BookResource` 保存条码号、书号、书名、作者、出版社、馆藏数、可借数、价格八项公共数据，派生类只增加页数、期号、访问地址等各自特有的成员；借阅天数与日罚金由派生类各自实现，这正是三种资源规则不同的落点。

4.3 组合关系与依赖关系

组合关系有两处：

1. `Library` 与资源集合、借阅人集合构成整体与部分关系。`Library` 的构造函数建立容器，析构函数负责释放，被组合对象的生命周期不长于整体。
2. `Borrower` 与 `LoanRecord` 构成整体与部分关系。借阅记录以值对象形式存放于 `vector<LoanRecord>` 中，随借阅人一同创建与销毁；删除借阅人时其借阅记录随之消失。

依赖关系体现在借还函数上：

```
bool Borrower::borrowResource(BookResource& resource, const Date& today,
std::string& message);
```

借阅人“使用”图书资源，但资源不是借阅人的成员，二者生命周期独立。函数以引用方式接收资源对象，在内部同时修改资源的可借数与借阅人的记录集合，因此这是典型的依赖关系而非组合关系。

4.4 多态的实现要点

多态在本项目中承担三项具体工作：

1. **同一容器存放不同类型**：`DataStore<BookResource>` 内部是 `std::vector<BookResource*>`，三种资源对象共存其中。
2. **同一调用得到不同输出**：`Library::searchResources` 收集匹配对象后统一写出 `os << *ptr`，实际调用的是各派生类自己的 `print`。
3. **同一规则得到不同参数**：借书时统一调用 `resource.maxBorrowDays()`，普通图书返回 30、期刊返回 7，调用方无需判断类型。

   支撑多态的语法是“基类声明纯虚函数 + 派生类用 `override` 重写 + 通过基类指针或引用调用”。

5 各文件详解

本章按 3.3 节的依赖顺序自下而上展开：先讲基础设施，再讲数据模型，最后讲业务与界面。

5.1 Utils.h 与 Utils.cpp

该模块集中了三个与业务无关的工具类。它们被其余所有文件使用，但不依赖任何业务类型，因此放在依赖链最底端。

5.1.1 头文件的组织方式

```
#pragma once
#include <string>
#include <vector>
#include <iostream>

class Text { /* 全静态成员函数 */ };
class Date { /* 日期值类型 */ };
class Console{ /* 控制台工具 */ };
```

语法点：`#pragma once` 是编译器层面的头文件防重复包含指令，作用等价于传统的 `#ifndef / #define / #endif` 组合，但无需自造宏名且不会因宏名冲突失效。
为什么这样组织：三个类都很小（合计约 80 行声明），且同属“通用工具”这一关注点。若拆成三个头文件，每个使用方都要写三行 `#include`，收益不抵成本；合并后 `#include "Utils.h"` 一行即可。需要强调的是，合并的只是**文件**，三个类仍然是三个独立的类型，职责边界没有合并。

5.1.2 Text 类：字符串处理

`Text` 的全部成员都是 `static`，调用形式为 `Text::trim(s)`，不需要创建对象。选择静态成员函数而不选全局函数的原因是：静态函数仍处于类作用域中，可通过类名归类，避免命名空间污染，同时满足“不使用大量全局函数”的要求。
区块一：按分隔符切分

```
std::vector<std::string> Text::split(const std::string& text, char sep) {
std::vector<std::string> parts;
std::string item;
std::istringstream iss(text);
while (std::getline(iss, item, sep)) {
parts.push_back(item);
}
if (!text.empty() && text.back() == sep) {
parts.push_back("");
}
return parts;
```

```
}
```

- **作用**：把一行数据记录按 `|` 拆成若干字段，供文件读取时逐字段赋值。
- **语法点**：`std::istringstream` 把字符串包装成输入流，`std::getline(stream, target, delim)` 以指定分隔符读取一段，比手写下标扫描更不易出错；`std::string::back()` 取末字符。
- **执行逻辑**：逐个读出字段压入 `vector`；`getline` 遇到“末尾紧跟分隔符”的情况会少产生一个空字段，因此末尾单独补一个空串，保证 `a||c` 与 `a||` 两种情况字段数都与书写一致。这一处理直接决定了记录格式的健壮性——派生类的扩展字段允许为空。

区块二：去首尾空白与格式清理

```
std::string Text::trim(const std::string& text) {
const std::string blanks = " \t\r\n";
std::size_t begin = text.find_first_not_of(blanks);
if (begin == std::string::npos) {
return "";
}
std::size_t end = text.find_last_not_of(blanks);
return text.substr(begin, end - begin + 1);
}

std::string Text::sanitize(const std::string& text) {
std::string result = trim(text);
for (std::size_t i = 0; i < result.size(); ++i) {
char ch = result[i];
if (ch == '|' || ch == ';' || ch == '\t' || ch == '\n' || ch == '\r') {
result[i] = ' ';
}
}
return result;
}
```

- **作用**：`trim` 消除用户输入两侧多余空格；`sanitize` 在写文件前把会破坏记录格式的字符替换为空格。
- **语法点**：`find_first_not_of` / `find_last_not_of` 按字符集合查找，返回值类型为 `std::size_t`，失败时返回常量 `std::string::npos`；`substr(pos, count)` 取子串；范围 `for` 改写为下标循环是为了在遍历中直接修改字符。
- **执行逻辑**：`sanitize` 先 `trim` 再去脏字符。若书名中含有 `|`，写入文件后该行字段会错位，读回时类型与字段全部对不上；在写入侧统一净化，是成本最低的防护。

区块三：安全数值转换与金额格式化

```
int Text::toInt(const std::string& text, int defaultValue) {
std::istringstream iss(trim(text));
int value = 0;
return (iss >> value) ? value : defaultValue;
}
```

```
std::string Text::money(double value) {
std::ostringstream oss;
oss.setf(std::ios::fixed);
oss.precision(2);
oss << value;
return oss.str();
}
```

- **作用**：把字符串转为数字，失败时返回调用方指定的默认值；把金额统一格式化为两位小数。
- **语法点**：流对象在条件表达式中会被转换为 `bool`（`explicit operator bool`），转换成功为真；`setf(std::ios::fixed)` 与 `precision(2)` 是 `<iomanip>` 的格式控制成员函数。
- **执行逻辑**：`toInt` 不抛异常、不改动调用现场，读取失败即回退默认值，使文件里某一字段损坏时程序仍能继续加载其余记录，属于“容错优先”的设计取舍。

5.1.3 Date 类：日期与天数差

区块一：三种构造函数

```
Date::Date() { // 默认构造：系统当天
std::time_t now = std::time(nullptr);
std::tm local;
std::tm* pointer = std::localtime(&now);
local = *pointer;
year_ = local.tm_year + 1900;
month_ = local.tm_mon + 1;
day   = local.tm  mday;
}

Date::Date(int year, int month, int day) : year_(year), month_(month), day_(day) {
if (!isValid()) { // 非法日期回退为当天
Date base;
*this = base;
}
}

Date::Date(const Date& other)
: year_(other.year_), month_(other.month_), day_(other.day_) {
}
```

- **作用**：分别支持“取当天”“按年月日指定”“由已有对象复制”三种创建方式。
- **语法点**：构造函数重载（参数表不同即可同名）；成员初始化列表 `: year_(year)` 在对象创建时直接初始化成员，比在函数体内赋值少一次默认构造；`tm_year` 以 1900 为基准、`tm_mon` 以 0 为基准，需分别加 1900 与 1。
- **执行逻辑**：重载构造函数在初始化列表完成后立即自检，若年月日非法则整体替换为当天值。这一“构造即保证合法”的做法，使后续所有使用 `Date` 的代码都不必再判空或判非法，把校验成本收敛到一处。
- **拷贝构造的必要性**：本类无堆资源，编译器生成的默认拷贝构造已足够，此处显式实现是为了

与实验要求一致，并明确表达“复制语义为逐成员复制”这一意图。

区块二：日期与整数序号互转

```
int Date::toSerial() const {
int year = year_, month = month_;
if (month <= 2) { year -= 1; month += 12; }
int era = (year >= 0 ? year : year - 399) / 400;
unsigned yearOfEra = static_cast<unsigned>(year - era * 400);
unsigned dayOfYear = static_cast<unsigned>(
(153 * (month + (month > 2 ? -3 : 9)) + 2) / 5 + day_ - 1);
unsigned dayOfEra = yearOfEra * 365 + yearOfEra / 4 - yearOfEra / 100 + dayOfYear;
return era * 146097 + static_cast<int>(dayOfEra) - 719468;
}
```

- **作用**：把人类可读的日期换算为“距 1970-01-01 的天数”，使日期比较与天数差退化为整数减法。
- **语法点**：`static_cast<unsigned>` 显式类型转换，避免有符号与无符号混算的隐式提升告警；整除运算 `/` 配合闰年修正项实现公历天数累计。
- **执行逻辑**：1、2 月被划归上一年的第 13、14 月，从而把闰日集中到年末，消除“2 月长度随年份变化”对累计公式的干扰；`era` 表示 400 年周期序号，每个周期固定 146097 天。`daysBetween` 直接返回两个序号之差，`addDays` 执行逆运算得到新日期，二者互为反函数，因此“借出日期 + 借期 = 应还日期”可精确到天而不受月份长度影响。

区块三：合法性与输出

```
bool Date::isValid() const {
if (year_ < 1900 || year_ > 2200) return false;
if (month_ < 1 || month_ > 12) return false;
return day   >= 1 && day   <= daysOfMonth(year  , month  );
}

std::string Date::toString() const {
std::ostringstream oss;
oss << std::setfill('0') << std::setw(4) << year   << '-'
<< std::setw(2) << month_ << '-' << std::setw(2) << day_;
return oss.str();
}
```

- **作用**：判断日期是否合法；把日期格式化为 `2026-10-29` 形式以保证文件可比对、可解析。
- **语法点**：`std::setw` 设定下一次输出的最小宽度，`std::setfill('0')` 设定填充字符，二者配合实现补零；`operator<<` 与 `operator>>` 被重载为友元函数，使 `Date` 对象可直接参与流操作。
- **执行逻辑**：`isValid` 按“年份范围 → 月份范围 → 当月天数”三级判断，`daysOfMonth` 内置闰年规则，2 月在闰年返回 29。补零格式是文件格式约定的一部分：记录按 `|` 切分后，日期字段固定 10 个字符，流提取时可稳定解析。

5.1.4 Console 类：控制台与输入校验

区块一：编码初始化

```
void Console::init() {
#if defined(_WIN32)
SetConsoleOutputCP(CP_UTF8);
SetConsoleCP(CP_UTF8);
#endif
}
```

- **作用**：把控制台输出与输入代码页同时切换为 UTF-8（65001），与源文件编码保持一致。
- **语法点**：`#if defined(_WIN32)` 条件编译，使同一份代码在非 Windows 平台不引用 `windows.h`；`SetConsoleOutputCP` / `SetConsoleCP` 为 Windows 控制台 API。
- **执行逻辑**：程序启动时调用一次。中文乱码的成因是“源文件字节序列的编码”与“控制台解释字节所用的代码页”不一致；编译期用 `-fexec-charset=UTF-8` 固定前者，运行期用本函数固定后者，两端一致即不乱码。

区块二：带范围校验的整数输入

```
int Console::readInt(const std::string& prompt, int low, int high) {
while (true) {
std::string value = readLine(prompt);
if (!value.empty() && Text::isDigits(value)) {
int number = Text::toInt(value, low - 1);
if (number >= low && number <= high) {
return number;
}
}
std::cout << " [输入无效] 请输入 " << low << " ~ " << high
<< " 之间的整数。\n";
}
}
```

- **作用**：读取一个落在指定闭区间内的整数，非法输入不返回、直接重新询问。
- **语法点**：`while (true)` 加 `return` 构成“校验—重试”循环；先判 `isDigits` 而不是直接用 `>>` 提取，是为了避免 `12abc` 被当成 12 接受。
- **执行逻辑**：三层过滤依次为“非空 → 全数字 → 落在区间”，任一不满足则打印提示并重新读取。所有菜单编号、数量、金额输入都经由本函数，因此非法输入在界面层就被拦截，业务层不再需要处理格式错误。

区块三：输入流结束的处理

```
std::string Console::readLine(const std::string& prompt) {
std::cout << prompt;
std::string line;
if (!std::getline(std::cin, line)) {
if (std::cin.eof()) {
throw ExitSignal();
```

```
}
std::cin.clear();
return "";
}
return Text::trim(line);
}
```

- **作用**：读取一行输入；当输入流结束（如重定向输入用尽）时抛出 `ExitSignal`，交由 `main` 统一收尾。
- **语法点**：`throw` 抛出自定义空类 `ExitSignal`，`main` 中以 `catch (const Console::ExitSignal&)` 捕获；`std::cin.clear()` 清除流状态位以便继续使用。
- **执行逻辑**：若在读取失败时简单返回空串，菜单循环会不断重试并占满 CPU；抛出信号让控制流直接跳回 `main`，保证退出前仍执行数据保存。这是把“异常”用作“非本地跳转”的典型场景——它不是错误，而是一种提前结束的请求。

5.2 DataStore.h

该文件只有 72 行，是本项目唯一的类模板，承担“批量对象管理”职责。

```
template <typename T>
class DataStore {
public:
DataStore() {}
~DataStore() { clear(); }

DataStore(const DataStore&) = delete;
DataStore& operator=(const DataStore&) = delete;

void add(T* item) { items_.push_back(item); }
std::size_t size() const { return items_.size(); }
T* at(std::size_t index) const { return items_[index]; }

std::size_t indexOf(const std::function<bool(const T*)>& predicate) const {
for (std::size_t i = 0; i < items_.size(); ++i) {
if (predicate(items_[i])) return i;
}
return notFound();
}

bool removeAt(std::size_t index) {
if (index >= items_.size()) return false;
delete items  [index];
items_.erase(items_.begin() + static_cast<long>(index));
return true;
}

private:
std::vector<T*> items_;
};
```

- **作用**：用一个 `vector` 存放同类型但不同派生类的对象指针，并统一负责释放；对外提供增加、按条件查找、按下标删除、遍历、清空五项能力。
- **语法点一：类模板**。`template <typename T>` 声明类型参数，`T` 在实例化时被替换为具体类型。本项目两处实例化为 `DataStore<BookResource>` 与 `DataStore<Borrower>`，编译器据此生成两份独立的类代码。模板的价值在于：若无模板，要么为两种对象各写一份几乎相同的容器类，要么退化为 `void*` 容器而丢失类型安全。
- **语法点二：`= delete`**。将拷贝构造与拷贝赋值声明为删除函数，任何复制容器的尝试在编译期即报错。原因是容器持有对象所有权并已在析构时 `delete`，一旦复制就会产生两个容器指向同一批对象，析构时二次释放导致程序崩溃。
- **语法点三：`std::function` 回调**。`indexOf` 接收 `std::function<bool(const T*)>`，把“判断条件”作为参数传入。这样容器不必知道 `BookResource` 有 `getCode()` 方法，查找条件由调用方提供，容器保持通用。
- **语法点四：指针容器与析构**。`std::vector<T*>` 存放指针本身，`std::vector` 析构不会释放所指对象，因此必须在 `removeAt` 与 `clear` 中显式 `delete`。
- **执行逻辑**：`add` 只把指针压入尾部，所有权随之转移给容器；`removeAt` 先 `delete` 再 `erase`，顺序不可颠倒——若先 `erase`，指针已从容器中移除，对象便再无人持有而形成内存泄漏；`clear` 在析构函数中被调用，保证程序结束或重新装载数据时不残留对象。

5.3 BookResource.h 与 BookResource.cpp

该文件包含 1 个抽象基类与 3 个派生类，是“馆藏资源”这一业务概念的完整定义。

5.3.1 基类的数据成员与访问控制

```
class BookResource {
public:
// 获取与修改接口（略）
virtual std::string typeName() const = 0;
virtual int maxBorrowDays() const = 0;
virtual void print(std::ostream& os) const = 0;
// ...
protected:
std::string code_; // 条码号，馆内唯一
std::string number_; // ISBN / ISSN
std::string title_; // 书名
std::string author_; // 作者
std::string publisher_; // 出版社
int total_; // 馆藏册数
int available_; // 可借册数
double price_; // 定价
};
```

- **作用**：把三类资源共有的八项数据集中到基类，避免在每个派生类中重复声明。

- **语法点：`protected` 访问权限**。`protected` 成员对类外不可见，但对派生类可见。因此 `Book::print` 可以直接读 `title_`，而 `ConsoleUI` 中只能写 `resource->getTitle()`。这既保护了数据完整性（外部无法绕过校验直接改 `available_`），又避免了派生类为访问父类数据而写大量 getter。
- **为什么把数据放在基类而不放派生类**：若将公共字段放入各派生类，`Library` 在只持有基类指针时无法统一读写，必须 `dynamic_cast` 逐一判断，多态随之失效。

5.3.2 构造函数、拷贝构造与虚析构

```
BookResource::BookResource(const std::string& code, const std::string& number,
const std::string& title, const std::string& author,
const std::string& publisher, int total, double price)
: code_(code), number_(number), title_(title), author_(author),
publisher_(publisher),
total_(total > 0 ? total : 1),
available_(total > 0 ? total : 1),
price_(price > 0.0 ? price : 0.0) {
}

BookResource::~BookResource() {
// 无堆资源；声明为虚函数是为了让 delete 基类指针时正确析构派生类部分
}
```

- **作用**：在创建对象时完成字段初始化，并保证初始状态合法。
- **语法点一：构造初始化列表**。`total_(total > 0 ? total : 1)` 使用三目运算符在初始化列表中直接完成取值修正，避免“先赋非法值再改”的中间状态。
- **语法点二：虚析构**。`virtual ~BookResource()` 是关键设计。容器中保存的是 `BookResource*`，执行 `delete ptr` 时，若析构函数非虚，编译器按静态类型只调用基类析构，派生类新增成员（如 `std::string category_`）不会被释放，形成内存泄漏。声明为虚后，`delete` 会按对象的实际类型逐层析构。
- **执行逻辑**：可借数初值等于馆藏总数，表示新入藏资源全部可借；后续借还操作由 `takeOne` 与 `giveOne` 维护两数之差。

5.3.3 借还状态维护

```
bool BookResource::takeOne() {
if (available_ <= 0) return false;
--available_;
return true;
}

void BookResource::giveOne() {
if (available  < total  ) ++available  ;
}
```

```
bool BookResource::changeTotal(int delta) {
int borrowed = getBorrowed(); // 必须先取出，再改 total_
int newTotal = total_ + delta;
if (newTotal < borrowed || newTotal > 9999) return false;
total_ = newTotal;
available_ = newTotal - borrowed;
return true;
}
```

- **作用**：借出与归还时维护“可借册数”，并支持增减馆藏总量。精简后界面不再提供“增减馆藏数量”的入口，`changeTotal` 作为模型层的册数调整接口保留，任何时候调用都自带合法性检查。
- **语法点**：`++` / `--` 前置自增自减；`getBorrowed()` 以 `total_ - available_` 计算已借数，把“已借”作为派生量而不单独存储，避免三份数据互相矛盾。
- **执行逻辑（关键）**：`changeTotal` 中必须先保存 `borrowed` 再修改 `total_`。若先执行 `total_ = newTotal`，随后的 `getBorrowed()` 会用新总量减旧可借数，得出错误的已借数，进而把可借数算错。这一顺序问题是实现过程中真实出现过的缺陷，修复方式即把派生量提前取出。
- **边界处理**：`takeOne` 在无可借副本时返回 `false` 而不减到负数；`giveOne` 在已达上限时不再增加，防止归还次数异常导致可借数超过总数。

5.3.4 国际标准号校验

```
bool BookResource::isValidStdNumber(const std::string& number) {
std::string digits;
for (std::size_t i = 0; i < number.size(); ++i) {
if (number[i] != '-' && number[i] != ' ') digits.push_back(number[i]);
}
int sum = 0;
if (digits.size() == 13) { // ISBN-13
for (int i = 0; i < 13; ++i) {
int value = digits[static_cast<std::size_t>(i)] - '0';
sum += (i % 2 == 0) ? value : value * 3;
}
return sum % 10 == 0;
}
if (digits.size() == 10) { /* ISBN-10 加权和对 11 取模 */ }
if (digits.size() == 8) { /* ISSN-8 加权和对 11 取模 */ }
return false;
}
```

- **作用**：验证用户输入的书号是否真实合法，非法书号拒绝录入。
- **语法点**：字符 `'0'` 到 `'9'` 的 ASCII 码连续，`digits[i] - '0'` 即得对应数值；`static std::size_t` 强制下标类型，消除容器下标的有符号无符号混用告警。
- **执行逻辑**：先剔除连字符与空格得到纯有效字符，再按长度分派到三种校验规则。ISBN-13 的规则是奇数位权重 1、偶数位权重 3，加权和能被 10 整除；ISBN-10 与 ISSN-8 的末位可为

`X`（代表 10），加权和能被 11 整除。三种规则共用同一入口，调用方无需关心具体是哪种书号。

- **设计取舍**：这里只校验校验位，不校验前缀与出版者号区间，因此“格式合法但不存在”的书号仍可通过。对课程项目而言，这一强度已足以体现数据合法性验证的设计意图。

5.3.5 序列化与工厂函数

```
std::string BookResource::toRecord() const {
std::ostringstream oss;
oss << typeTag() << '|' << Text::sanitize(code_) << '|'
<< Text::sanitize(number_) << '|' << Text::sanitize(title_) << '|'
<< Text::sanitize(author_) << '|' << Text::sanitize(publisher_) << '|'
<< total_ << '|' << available_ << '|' << price_ << '|' << extraData();
return oss.str();
}

BookResource* BookResource::fromRecord(const std::string& line) {
std::vector<std::string> fields = Text::split(line, '|');
if (fields.size() < 10) return nullptr;

BookResource* resource = nullptr;
if (fields[0] == "BOOK") resource = new Book();
else if (fields[0] == "MAG") resource = new Magazine();
else if (fields[0] == "EBOOK") resource = new EResource();
else return nullptr;

resource->code_ = fields[1]; // 静态成员函数可访问保护成员
// ... 逐字段回填
if (!resource->loadExtra(fields[9])) { delete resource; return nullptr; }
return resource;
}
```

- **作用**：把对象写成一行文本；从一行文本还原出具体派生类对象。
- **语法点一：虚函数参与序列化**。`typeTag()` 与 `extraData()` 是虚函数，同一份 `toRecord` 代码为三种资源写出不同的类型标记与扩展字段，无需 `switch` 判断类型。
- **语法点二：静态成员函数访问保护成员**。`fromRecord` 是 `BookResource` 的静态成员函数，因此可以直接给 `resource->code_` 赋值；但 `loadExtra` 是虚函数，会按实际类型解析到派生类版本。
- **语法点三：工厂模式**。由类型标记决定 `new` 哪个类，返回基类指针。这是“保存派生类对象”这一问题的标准解法：文件里无法保存 vtable，只能保存类型标记，读取时依据标记重新构造。
- **执行逻辑**：写入时先类型标记、再公共字段、最后扩展字段；读取时按同样顺序反向解析。对字段数不足、类型标记未知、扩展字段解析失败三种异常均返回空指针，由调用方跳过该行，保证单条记录损坏不影响整体装载。

5.3.6 三个派生类的差异

```
class Book : public BookResource {
public:
std::string typeName() const override { return "普通图书"; }
int maxBorrowDays() const override { return 30; }
double finePerDay() const override { return 0.20; }
void print(std::ostream& os) const override;
std::string extraData() const override;
private:
int pages_;
std::string category_;
};
```

- **作用**：以最小的代码量表达三种资源在业务规则上的差异。
- **语法点一：`public` 继承**。公有继承表示“是一个”的语义，派生类对象可被基类指针指向。
- **语法点二：`override` 说明符**。显式声明该函数重写基类虚函数，若函数签名写错（如漏写 `const`）编译器立即报错，避免“以为重写了、实际是新增函数”的隐蔽缺陷。
- **语法点三：类内定义虚函数**。函数体只有一行时直接在类内定义，编译器会自动将其视为内联候选，减少函数调用开销。
- **三种资源的规则差异**：

资源类型	借期	日罚金	特有字段	显示特征
普通图书	30 天	0.20 元	页数、分类	显示 ISBN、页数、分类
期刊杂志	7 天	0.50 元	期号、出版周期	显示 ISSN、期号、周期
电子资源	3 天	0.10 元	访问地址、文件大小	显示授权份数、地址、大小

- **扩展字段的实现**：`extraData()` 把派生类特有成员拼成 `字段 1;字段 2` 形式追加在记录末尾；`loadExtra()` 按分号拆开回填。基类不关心其含义，实现“基类定框架、派生类填内容”。

5.4 Borrower.h 与 Borrower.cpp

该文件包含 `LoanRecord`、`Borrower`、`Student`、`Teacher` 四个类，负责“谁在借、借了什么、什么时候该还”。

5.4.1 LoanRecord：借阅记录

```
int LoanRecord::overdueDays(const Date& today) const {
```

```
int days = today.daysBetween(dueDate_);
return days > 0 ? days : 0;
}

std::string LoanRecord::toString(const Date& today) const {
std::ostringstream oss;
oss << code_ << "  《" << title_ << "》  借出  " << borrowDate_.toString()
<< "  应还  " << dueDate_.toString();
int overdue = overdueDays(today);
if (overdue > 0) oss << "  【已逾期  " << overdue << "  天】";
else oss << "  【未逾期】";
return oss.str();
}
```

- **作用**：记录一次借阅的条码号、书名、借出日期与应还日期，并据此判断是否逾期。
- **语法点**：`const` 成员函数承诺不修改对象状态，因此可在 `const Borrower&` 上调用；`Date` 成员以值对象形式嵌入，随记录一同复制，无需手动管理。
- **执行逻辑**：逾期天数由“今天减应还日期”得出，未逾期时统一返回 0，使上层判断只需比较 `> 0`。这里用“差额取正”而非布尔标志，是因为罚金需要天数本身。
- **设计说明**：记录中冗余保存书名（`title_`），是为了在借阅人详情中直接显示书名而不必再查资源表；这是以少量冗余换取显示效率与解耦的常见做法。

5.4.2 Borrower：借书与还书

区块一：借书的三级判定

```
bool Borrower::borrowResource(BookResource& resource, const Date& today,
std::string& message) {
if (getLoanCount() >= maxBooks_) {
message = "借阅失败：已达最大借阅数量  ...";
return false;
}
if (!resource.isAvailable()) {
message = "借阅失败：《" + resource.getTitle() + "》暂无可借副本。";
return false;
}
for (std::size_t i = 0; i < loans_.size(); ++i) {
if (loans_[i].getCode() == resource.getCode()) {
message = "借阅失败：该资源您已借阅且尚未归还。";
return false;
}
}
if (!resource.takeOne()) { message = "借阅失败：馆藏数量不足。"; return false; }

Date dueDate = today.addDays(resource.maxBorrowDays());
loans_.push_back(LoanRecord(resource.getCode(), resource.getTitle(), today, dueDate));
// ...  组装成功消息
```

```
return true;
}
```

- **作用**：在满足全部前置条件的前提下完成借出，并同时更新图书与借阅人双方状态。
- **语法点一：引用参数 `BookResource&`**。以引用接收资源对象，函数内修改的是调用方的同一对象，可借数减一才会真正生效。若改为值传递，修改的是副本，借出后馆藏数不变——这是本节最重要的语法点。
- **语法点二：`const Date&` 与 `std::string&` 的搭配**。日期只读，用常引用避免拷贝；`message` 需回传结果，用非 const 引用作为输出参数。
- **语法点三：`std::to_string`**。把整数转为字符串用于消息拼接。
- **执行逻辑**：判定顺序依次为“借阅人额度 → 图书可借数 → 是否重复借阅 → 扣减可借数”，前三级失败均不改变任何状态，保证操作的原子性；全部通过后才计算应还日期并写入记录。应还日期由 `maxBorrowDays()` 决定，该调用是多态分发点：借同一本书的不同类型资源，得到的借期不同。
- **依赖关系的体现**：本函数同时读写 `Borrower` 的数据成员与外部传入的 `BookResource` 对象，二者无所有权关系，正是依赖关系。

区块二：还书与罚款计算

```
bool Borrower::returnResource(BookResource& resource, const Date& today,
std::string& message, double& fine) {
fine = 0.0;
for (std::size_t i = 0; i < loans_.size(); ++i) {
if (loans_[i].getCode() != resource.getCode()) continue;

int overdue = loans_[i].overdueDays(today);
if (overdue > 0) {
fine = overdue * resource.finePerDay() * fineDiscount();
}
resource.giveOne();
loans_.erase(loans_.begin() + static_cast<long>(i));
// ... 组装消息
return true;
}
message = "归还失败：该借阅人名下没有此资源的借阅记录。";
return false;
}
```

- **作用**：删除借阅记录、恢复可借数、按逾期天数结算罚款。
- **语法点一：迭代器算术**。`loans_.begin() + i` 得到指向第 i 个元素的迭代器，`erase` 接受迭代器而非下标；`static_cast<long>` 用于消除 `size_t` 到 `difference_type` 的符号转换告警。
- **语法点二：引用输出参数 `double& fine`**。函数只有一个返回值，已用于表示成功与否，罚款额通过额外输出参数带回，避免为“结果 + 数据”而定义结构体。
- **语法点三：虚函数参与计算**。`resource.finePerDay()` 取该资源的日罚金，`fineDiscount()` 取该借阅人的折扣系数——一次乘法同时用到了资源与借阅人两条继承链

的多态。

- **执行逻辑**：先按条码号定位记录，再算逾期天数，最后依次“计价 → 恢复可借数 → 删除记录”。删除使用 `erase` 而非把记录标记为已还，原因是本项目只关心当前在借状态，历史记录由借阅人删除前的数据文件承载；这一取舍减少了状态字段与遍历判断。
- **罚款公式**：`罚款 = 逾期天数 × 资源日罚金 × 借阅人折扣系数`。教师系数 0.5，学生系数 1.0，因此同一本书逾期同样天数，教师罚款为学生的一半。

5.4.3 派生类 Student 与 Teacher

```
Student::Student(const std::string& id, const std::string& name,
const std::string& dept, const std::string& major,
const std::string& className)
: Borrower(id, name, dept, 5), // 学生默认最多借 5 册
major_(major.empty() ? "未填写" : major),
className_(className.empty() ? "未填写" : className) {
}

class Teacher : public Borrower {
public:
double fineDiscount() const override { return 0.5; } // 罚款减半
void printExtra(std::ostream& os) const override;
private:
std::string title ;
};
```

- **作用**：表达两类借阅人在额度与罚款政策上的差异。
- **语法点一：派生类构造函数调用基类构造函数**。初始化列表中写 `Borrower(id, name, dept, 5)`，先构造基类子对象再初始化自身成员。若省略，编译器会调用基类的默认构造函数，导致 `id_`、`name_` 等字段为空。
- **语法点二：额度由派生类固定**。学生传 5、教师传 10，`maxBooks_` 仍存放在基类，`Borrower::borrowResource` 中的额度判断无需区分身份。
- **语法点三：模板方法模式**。`Borrower::print` 负责输出公共字段与借阅明细，中途调用纯虚函数 `printExtra`，由派生类补充专业班级或职称。这样公共输出逻辑只写一次，派生类只实现差异部分。
- **执行逻辑**：`Teacher::fineDiscount` 返回 0.5，直接改变 5.4.2 中罚款公式的结果；不需要在还书流程中写 `if (身份 == 教师)`，规则差异被封装在类内。

5.5 Library.h 与 Library.cpp

`Library` 是业务层核心，418 行，包含集合管理、业务规则、统计与文件读写四组功能。

5.5.1 组合容器与查找

```
class Library {
public:
explicit Library(const std::string& dataDir);
std::size_t resourceCount() const { return resources_.size(); }
BookResource* findResource(const std::string& code) const;
// ...
private:
std::string dataDir_;
DataStore<BookResource> resources_;
DataStore<Borrower> readers_;
};
```

- **作用**：以两个模板容器持有全部资源与借阅人，构成“整体—部分”关系。
- **语法点**：`DataStore<BookResource>` 中的模板实参是**类名**而非指针类型，指针由模板内部生成，因此调用方写 `resources_.add(new Book(...))` 即可。
- **执行逻辑**：`Library` 构造时创建数据目录，析构时两个容器自动 `clear()`，全部堆对象随之释放；`Library` 自身禁止拷贝，防止两个图书馆对象共享同一批资源。

查找函数：

```
BookResource* Library::findResource(const std::string& code) const {
std::size_t index = resources_.indexOf([&code](const BookResource* item) {
return item->getCode() == code;
});
return index == DataStore<BookResource>::notFound()
? nullptr : resources_.at(index);
}
```

- **作用**：按条码号定位资源对象，是删除、借书、还书的公共前置步骤。
- **语法点一：Lambda 表达式**。`[&code](const BookResource* item) { ... }` 中 `[&code]` 以引用方式捕获外部变量，`item` 为形参；Lambda 被隐式转换为 `std::function<bool(const BookResource*)>`，与容器接口对接。
- **语法点二：`notFound()` 哨兵值**。容器用 `static_cast<std::size_t>(-1)` 表示“未找到”，避免与合法下标 0 混淆。
- **执行逻辑**：线性扫描容器，命中即返回下标；未命中返回空指针。数据量在千条以内时线性查找足够，若数据规模上升，可按条码号建立 `std::map` 索引，接口保持不变。

5.5.2 业务规则：删除保护

```
bool Library::removeResource(const std::string& code, std::string& message) {
std::size_t index = resources_.indexOf(/* 按条码号匹配 */);
if (index == DataStore<BookResource>::notFound()) { /* 未找到 */ }
```

```
BookResource* resource = resources_.at(index);
if (resource->getBorrowed() > 0) {
message = "删除失败：该资源还有 " + std::to_string(resource->getBorrowed())
+ " 册在借，请先归还。";
return false;
}
std::string title = resource->getTitle();
resources_.removeAt(index);
message = "删除成功：《" + title + "》已从馆藏移除。";
return true;
}
```

- **作用**：删除馆藏资源，并在有副本在借时拒绝删除。
- **语法点**：先取出 `title` 再调用 `removeAt`，因为 `removeAt` 会 `delete` 对象，之后访问 `resource->getTitle()` 属于悬垂指针访问。
- **执行逻辑**：`getBorrowed() > 0` 即“总数大于可借数”，说明有副本在外；此时删除会让借阅记录指向不存在的资源，因此必须拦截。`removeReader` 中同理，借阅人尚有未还图书时拒绝删除。两条保护规则共同维持“借阅记录必然指向存在的资源与存在的借阅人”这一数据一致性约束。

5.5.3 借还业务的转发

```
bool Library::borrowBook(const std::string& readerId, const std::string& code,
std::string& message) {
Borrower* reader = findReader(readerId);
if (reader == nullptr) { message = "借阅失败：未找到借阅人 ..."; return false; }

BookResource* resource = findResource(code);
if (resource == nullptr) { message = "借阅失败：未找到资源 ..."; return false; }

Date today = Date::today();
return reader->borrowResource(*resource, today, message); // 传引用，体现依赖
}
```

- **作用**：把界面的“借书请求”翻译为对象间协作：先查人、再查书，最后交给借阅人自己完成判定。
- **语法点**：`*resource` 对指针解引用得到对象引用，与 `Borrower::borrowResource(BookResource&, ...)` 的形参类型匹配；基类指针解引用后仍是基类引用，函数内部的多态调用依然有效。
- **执行逻辑**：`Library` 只做定位与转发，不重复实现额度、库存、重复借阅等判断。这些判断属于借阅人自身的职责，放在实体类内可保证“无论从哪个入口借书，规则都相同”。这是“信息专家模式”的体现：谁拥有数据，谁负责操作数据。

5.5.4 借阅清单的双模式输出

```
void Library::listLoans(std::ostream& os, bool onlyOverdue) const {
Date today = Date::today();
std::size_t total = 0, overdueCount = 0;

for (std::size_t i = 0; i < readers_.size(); ++i) {
const Borrower* reader = readers_.at(i);
const std::vector<LoanRecord>& loans = reader->getLoans();
bool printedHeader = false;
for (std::size_t j = 0; j < loans.size(); ++j) {
int days = loans[j].overdueDays(today);
if (days > 0) ++overdueCount;
++total;
if (onlyOverdue && days <= 0) continue;
if (!printedHeader) { /* 打印借阅人标题 */ printedHeader = true; }
os << " " << loans[j].toString(today);
if (days > 0) { /* 追加应缴罚款 */ }
os << '\n';
}
}
// 根据 onlyOverdue 输出不同的汇总语
}
```

- **作用**：一个函数同时支撑“在借记录”与“逾期清单”两个菜单项。
- **语法点一：`std::ostream&` 抽象输出目标**。参数类型为输出流引用，实际可以传 `std::cout`，也可以传文件流或字符串流，函数无需修改即可复用。
- **语法点二：延迟打印标题**。`printedHeader` 标志确保只有存在待显示记录时才输出借阅人标题，避免逾期模式下出现“有标题无内容”的空分组。
- **执行逻辑**：无论哪种模式都完整遍历一次，统计量 `total` 与 `overdueCount` 始终准确；仅显示环节按 `onlyOverdue` 过滤。相比写两个遍历函数，此做法把“统计”与“过滤显示”解耦在一处，维护时只需改一个地方。
- **罚款估算**：逾期行会反查资源对象取 `finePerDay()`，再乘以借阅人折扣，给出“若今天归还应缴多少”，与还书时的实算公式完全一致，避免两处口径不一。

5.5.5 文件持久化

区块一：装载

```
bool Library::loadAll() {
bool resourceFileFound = false;

std::ifstream resourceInput(pathOf("resources.txt"));
if (resourceInput.is_open()) {
resourceFileFound = true;
std::string line;
```

```
while (std::getline(resourceInput, line)) {
line = Text::trim(line);
if (line.empty() || line[0] == '#') continue;
BookResource* resource = BookResource::fromRecord(line);
if (resource == nullptr) continue;
if (findResource(resource->getCode()) != nullptr) {
delete resource; continue; // 条码号重复，丢弃新对象
}
resources .add(resource);
}
resourceInput.close();
}
// 同理装载 readers.txt，再装载 loans.txt
if (!resourceFileFound) { createSampleData(); saveAll(); }
return true;
}
```

- **作用**：启动时把三个数据文件还原为内存对象；首次运行时生成演示数据。
- **语法点一：`std::ifstream` 与 `std::getline`**。文件流以只读方式打开，逐行读取；`is_open()` 判断打开是否成功，文件不存在时不会抛异常，只返回失败状态。
- **语法点二：对象重建的顺序**。必须先装载资源与借阅人，再装载借阅记录，因为 `LoanRecord` 需要按 `readerId` 挂到已存在的借阅人上。
- **执行逻辑**：跳过空行与以 `#` 开头的注释行；`fromRecord` 返回空表示该行损坏，跳过而不中断；条码号重复时 `delete` 新对象，防止容器内出现两个同号资源。`resourceFileFound` 用于区分“文件不存在（首次运行）”与“文件存在但内容为空（用户删光了馆藏）”，只有前者才生成演示数据，避免用户清空后重启又被自动填满。

区块二：保存

```
bool Library::saveAll() const {
std::ofstream resourceOutput(pathOf("resources.txt"));
if (!resourceOutput.is_open()) return false;
resourceOutput << "# 资源: 类型|条码号|书号|书名|作者|出版社|总数|可借|价格|扩展\n";
resources_.forEach([&](const BookResource* item) {
resourceOutput << item->toRecord() << '\n';
});
resourceOutput.close();

// readers.txt、loans.txt 同法写出
return true;
}
```

- **作用**：把内存中的全部对象写回文本文件。
- **语法点一：`std::ofstream` 覆盖写**。以默认模式打开即截断原文件，无需先删除。
- **语法点二：`forEach` 与 Lambda 的组合**。容器提供遍历接口，Lambda 提供逐元素动作，二者组合替代显式下标循环，且遍历过程中不暴露底层 `vector`。
- **语法点三：虚函数参与写文件**。`item->toRecord()` 是基类方法，但内部调用 `typeTag()`

与 `extraData()` 两个虚函数，因此同一行代码为三种资源写出三种格式。

- **执行逻辑**：三个文件按“资源 → 借阅人 → 借阅记录”顺序写出，与装载顺序对应。任一文件打开失败立即返回 `false`，由 `main` 提示用户检查目录权限。

5.5.6 演示数据生成

```
void Library::createSampleData() {
resources_.clear();
readers_.clear();

resources_.add(new Book("B001", "9787115428028", "C++ Primer（第 5 版）",
"Stanley B. Lippman", "电子工业出版社", 3, 128.00, 848, "计算机"));
resources_.add(new Magazine("M001", "1004194X", "计算机学报",
"中国科学院", 5, 30.00, 12, "月刊"));
resources_.add(new EResource("E001", "9787115546081", "C++标准库（电子版）",
"Nicolai M. Josuttis", 2, 89.00,
"http://lib.swust.edu.cn/ebook/001", 25));
// ... 其余资源与借阅人

BookResource* demo = findResource("B002");
Borrower* reader = findReader("20250001");
if (demo != nullptr && reader != nullptr && demo->takeOne()) {
Date today = Date::today();
reader->addLoan(LoanRecord(demo->getCode(), demo->getTitle(),
today.addDays(-40), today.addDays(-10)));
}
}
```

- **作用**：首次运行时构造 7 种资源、3 位借阅人与 1 条逾期记录，使程序可直接演示。
- **语法点一：`new` 与容器所有权的转移**。`new Book(...)` 返回派生类指针，作为基类指针传入 `add`，所有权随之交给容器；此后调用方不再 `delete`。
- **语法点二：多态容器存放异类对象**。三次 `add` 传入三种不同派生类指针，容器统一按 `BookResource*` 保存。
- **执行逻辑**：末尾刻意构造一条借出日期为 40 天前、应还日期为 10 天前的记录，并调用 `takeOne()` 保持库存数量一致。这样首次演示即可看到逾期状态与罚款计算，无需手工造数据。

5.5.7 数据目录的定位

```
std::string Library::defaultDataDir(const char* argv0) {
namespace fs = std::filesystem;
fs::path exePath = fs::absolute(fs::path(argv0 != nullptr ? argv0 : "Library.exe"));
fs::path exeDir = exePath.parent  path();

if (fs::exists(exeDir / "build.bat")) { // exe  就在工程根
```

```
return (exeDir / "data").string();
}
fs::path projectRoot = exeDir.parent_path(); // exe 在 build 等子目录里
if (!projectRoot.empty() && fs::exists(projectRoot / "build.bat")) {
return (projectRoot / "data").string();
}
return (exeDir / "data").string(); // 工程被拆散时：数据放 exe 旁边
}
```

- **作用**：把“相对路径 `data`”换成“确定的一份 `data`”。无论从工程根启动、从 `build` 目录启动，还是直接双击 `build\Library.exe`，都读写同一个 `data` 目录。
- **语法点一：`std::filesystem::absolute`**。入参是 `argv[0]` 这种可能是相对路径的字符串，`absolute` 以当前工作目录为基准补全为绝对路径，之后 `parent_path()` 才能稳定拿到 exe 所在目录。
- **语法点二：局部命名空间别名**。函数内写 `namespace fs = std::filesystem;`，只在函数体内生效，既不污染全局，又免去反复书写长名字。这是“别名”而非 `using namespace`，不会把整个命名空间的名字都倒进当前作用域。
- **语法点三：路径拼接用 `/`**。`exeDir / "data"` 由 `std::filesystem::path::operator/` 完成，自动处理分隔符与结尾斜杠，`(exeDir / "data").string()` 再转回 `std::string` 交给 `Library` 构造函数。用字符串相加 `exeDir + "/data"` 在 Windows 反斜杠路径下容易拼出 `\\` 或漏分隔符。
- **执行逻辑**：三级回退。第一级看 exe 同级有没有 `build.bat`（说明 exe 就在工程根）；第二级看上一级；两级都不满足（例如把 exe 单独拷到别处答辩）则把数据放在 exe 旁边，保证程序永远能跑起来，而不是写到一个不存在的路径。
- **为什么需要它**：相对路径的基准是“进程的当前工作目录”，而工作目录由启动方式决定——双击 exe 时是 exe 所在目录，从 VSCode 任务或终端启动时是终端当前目录。不加处理就会出现两份 `data`，且互不同步。详见 9.3 节。

5.6 ConsoleUI.h 与 ConsoleUI.cpp

界面层 251 行，只做三件事：显示菜单、收集输入、调用业务函数并打印返回的消息。菜单由原来的 15 项精简为 11 项，对应的操作函数也从 12 个减到 7 个（另有 2 个录入辅助函数负责创建派生类对象）。

5.6.1 主循环与菜单分派

```
void ConsoleUI::mainMenu() const {
Console::title("图书借阅管理系统");
std::cout << " 【馆藏】 1 关键字查询 2 添加资源 3 删除资源\n"
<< " 【读者】 4 添加借阅人 5 删除借阅人\n"
<< " 【借还】 6 办理借书 7 办理还书 8 在借记录 9 逾期清单\n"
<< " 【统计】10 馆藏与借阅统计\n"
<< " 0 保存并退出\n";
```

```
Console::line('-', 62);
std::cout << "  馆藏  " << library_.resourceCount() << "  种  |  借阅人  "
<< library_.readerCount() << "  人\n";
Console::line('-', 62);
}

void ConsoleUI::run() {
while (true) {
Console::clearScreen();
mainMenu();
int choice = Console::readInt("请输入操作编号（0-10）：", 0, 10);
switch (choice) {
case 1: doSearchResource(); break;
case 2: doAddResource(); break;
case 3: doRemoveResource(); break;
case 4: doAddReader(); break;
case 5: doRemoveReader(); break;
case 6: doBorrow(); break;
case 7: doReturn(); break;
// case 8 / 9 / 10  直接调用业务层输出函数
case 0: return;
default: break;
}
}
}
```

- **作用**：程序的交互骨架，单层菜单循环直至用户选择退出。
- **语法点一：`switch` 分派**。`choice` 为整型，`case` 标签为编译期常量；每个分支末尾 `break` 防止贯穿执行。
- **语法点二：`return` 退出循环**。在 `case 0` 中直接 `return`，比设置循环标志再判断更简洁。
- **执行逻辑**：每轮先清屏再重绘菜单，保证界面不堆积历史输出；`readInt` 已把输入限制在 0~10 之间，10 个功能编号加 `0` 退出正好把 `switch` 填满，`default` 分支实际不会被执行，保留它是为了把 `switch` 写完整、避免漏分支时落入未定义行为。

5.6.2 输入校验链：以新增资源为例

```
BookResource* ConsoleUI::inputNewResource() const {
std::cout << "  类型：1  普通图书  2  期刊杂志  3  电子资源  0  取消\n";
int type = Console::readInt("  请选择（0-3）：", 0, 3);
if (type == 0) return nullptr;

std::string code = Console::readText("  条码号（如  B010）：");
if (library_.findResource(code) != nullptr) {
std::cout << " [提示]  条码号已存在，添加终止。\n";
return nullptr;
}
```

```
std::string number;
while (true) {
number = Console::readLine("  书号（ISBN-13 / ISBN-10 / ISSN-8）: ");
if (BookResource::isValidStdNumber(number)) break;
std::cout << " [校验失败]  校验位不正确，请重新输入  ...\n";
}
// ...  依次读取书名、作者、出版社、数量、价格，最后按类型创建对象
return new Book(code, number, title, author, publisher, total, price, pages, category);
}
```

- **作用**：按用户输入创建一个具体的派生类对象，返回基类指针。
- **语法点一：返回基类指针**。函数返回类型为 `BookResource*`，三个分支分别 `new` 出 `Book`、`Magazine`、`EResource`，调用方无需知道实际类型。
- **语法点二：校验前置**。条码号重复检测在读取后立即进行，避免用户填完全部字段才被拒绝；书号校验放在 `while` 循环中，不通过则原地重输。
- **语法点三：取消语义**。选择 0 或条码号重复时返回 `nullptr`，调用方据此终止流程，这是以空指针表达“无结果”的常规做法。
- **执行逻辑**：先读公共字段，再按类型读特有字段（图书读页数与分类、期刊读期号与周期、电子资源读地址与大小），最后一次性构造对象。这样每个派生类的必填项都在创建前收集完毕，不存在“先建对象再补字段”的不完整状态。

5.6.3 借还操作的界面实现

```
void ConsoleUI::doBorrow() const {
Console::clearScreen();
Console::title("办理借书");
std::string id = Console::readText("借阅人学号 / 工号: ");
Borrower* reader = library_.findReader(id);
if (reader != nullptr) {
std::cout << " " << reader->roleName() << " " << reader->getName()
<< "  已借  " << reader->getLoanCount() << " / "
<< reader->getMaxBooks() << "  册\n";
}
std::string code = Console::readText("资源条码号: ");
std::string message;
bool ok = library_.borrowBook(id, code, message);
std::cout << (ok ? "\n [成功] " : "\n [失败] ") << message << '\n';
Console::pause();
}
```

- **作用**：完成一次借书的交互，并在失败时显示业务层给出的具体原因。
- **语法点一：多态读取身份信息**。`reader->roleName()` 由派生类实现，界面无需判断是学生还是教师。
- **语法点二：三目运算符控制提示前缀**。`ok ? "[成功]" : "[失败]"` 使成功与失败共用一条输出语句。

- **语法点三：结果消息由业务层生成**。界面只负责显示 `message`，不含任何业务判断；若把“额度不足”“无可借副本”等判断写在界面里，业务规则就被分散到两处，后续修改容易遗漏。
- **执行逻辑**：先查人并显示当前额度（查不到也继续，由 `borrowBook` 给出统一的失败原因），再读条码号，最后调用业务函数并显示其返回值。

5.7 main.cpp

```
int main(int argc, char* argv[]) {
Console::init(); // 统一 UTF-8，防止中文乱码

// 默认由可执行文件位置推断工程目录，也可用第一个命令行参数显式指定
std::string dataDir = (argc > 1) ? std::string(argv[1])
: Library::defaultDataDir(argv[0]);
Library library(dataDir);
library.loadAll(); // 从 data 目录读取对象数据

int exitCode = 0;
try {
ConsoleUI ui(library);
ui.run();
} catch (const Console::ExitSignal&) {
std::cout << "\n 检测到输入结束，正在保存数据……\n";
}

if (library.saveAll()) {
std::cout << "数据已保存到 " << dataDir << " 目录。感谢使用，再见！\n";
} else {
std::cout << "[警告] 数据保存失败，请检查 " << dataDir
<< " 目录是否可写。\n";
exitCode = 1;
}
return exitCode;
}
```

- **作用**：程序唯一的入口，负责对象装配与生命周期管理，共 38 行，不含任何业务逻辑。
- **语法点一：栈上对象**。`Library` 与 `ConsoleUI` 均以局部对象形式创建，离开作用域自动析构，无需手工 `delete`。
- **语法点二：引用成员**。`ConsoleUI` 持有一个 `Library&`，构造时绑定，界面因此直接操作同一个图书馆对象而非副本。
- **语法点三：`try / catch`**。捕获 `ExitSignal` 后仍继续执行后续保存逻辑，实现“异常退出但数据不丢”。
- **语法点四：返回退出码**。保存失败返回 1，供脚本或 IDE 判断执行结果。
- **语法点五：带参数的 `main`**。形参 `int argc, char* argv[]` 接收命令行参数，`argv[0]` 恒为程序自身的路径。本程序用它做两件事：把 `argv[0]` 交给 `defaultDataDir` 推断工程

目录；若用户额外传入一个参数（`argc > 1`），则把该参数当作数据目录使用。这也是唯一需要改 `main` 签名的地方，业务代码不受影响。

- **执行逻辑**：初始化编码 → 建对象 → 装数据 → 跑界面 → 存数据 → 返回。五个步骤的顺序不可调换：编码必须在任何输出之前设置，装载必须在界面之前完成，保存必须在界面结束之后执行。

6 关键流程说明

本章以时序方式描述五条主要流程，说明“用户操作 → 函数调用链 → 状态变化”的完整路径。

6.1 启动与数据装载

```
main()
└── Console::init() 设置 UTF-8 代码页
└── Library::defaultDataDir(argv[0]) 由 exe 位置推断工程根，拼出 <工程根>/data
└── Library library(dataDir) 创建数据目录（std::filesystem）
└ library.loadAll()
├── 打开 data/resources.txt
│ └── 逐行 BookResource::fromRecord()
│ ├── 按类型标记 new Book / Magazine / EResource
│ ├── 回填公共字段（基类静态成员函数访问保护成员）
│ └── loadExtra() 解析派生类特有字段
├── 打开 data/readers.txt → Borrower::fromRecord() → 挂入 readers_
├── 打开 data/loans.txt → LoanRecord::parse() → 挂到对应借阅人
└── 若资源文件不存在 → createSampleData() → saveAll()
└ ConsoleUI ui(library); ui.run()
```

关键点有三：一是三类文件的装载顺序固定为资源、借阅人、借阅记录，因为后者依赖前两者已存在；二是任一记录损坏只跳过该行，不中断整体装载；三是首次运行检测依据是“资源文件是否存在”，而非“容器是否为空”。

6.2 借书流程

```
用户输入 11 → 输入学号、条码号
└ ConsoleUI::doBorrow()
└ Library::borrowBook(readerId, code, message)
├── findReader(readerId) ——— 失败：返回“未找到借阅人”
├── findResource(code) ——— 失败：返回“未找到资源”
└ reader->borrowResource(*resource, today, message)
├── 额度检查：getLoanCount() >= maxBooks_ ？
├── 库存检查：resource.isAvailable() ？
├── 重复检查：该条码号是否已在 loans_ 中？
├── resource.takeOne() 可借数 -1
├ dueDate = today.addDays(resource.maxBorrowDays())
└ loans .push back(LoanRecord(...))
```

失败路径均在任何状态改变之前返回，因此不存在“扣了库存却没生成记录”的中间态。`maxBorrowDays()` 是多态分派点：同一段代码对普通图书取 30、期刊取 7、电子资源取 3。

6.3 还书与罚款流程

```
用户输入 12 → 输入学号、条码号
└─ ConsoleUI::doReturn()
└─ Library::returnBook(...)
└─ reader->returnResource(*resource, today, message, fine)
├── 在 loans_ 中按条码号定位记录 ——— 未找到：返回失败
├── overdue = today.daysBetween(dueDate) 取正值
├─ fine = overdue × resource.finePerDay() × fineDiscount()
├── resource.giveOne() 可借数 +1
└── loans_.erase(该记录)
```

罚款计算的三个因子分别来自：日期类（天数）、资源类（日罚金，虚函数）、借阅人类（折扣，虚函数）。这一条公式同时贯穿三条继承链，是“多态支撑业务规则差异化”的典型例证。

举例：借阅人张三（学生，系数 1.0）借《数据结构》，应还日期 10 天前，图书日罚金 0.20 元，则罚款 = 10 × 0.20 × 1.0 = 2.00 元；若换作教师（系数 0.5）借同一本书，罚款为 1.00 元。

6.4 新增馆藏流程

```
用户输入 3
└─ ConsoleUI::inputNewResource()
├── 选择类型（1/2/3）
├── 读条码号 → library_.findResource(code) 判重
├── 读书号 → BookResource::isValidStdNumber() 校验位验证（循环直至通过）
├── 读公共字段：书名、作者、出版社、数量、价格（均带范围校验）
├── 按类型读特有字段：页数/分类、期号/周期、地址/大小
└─ return new Book(...) / new Magazine(...) / new EResource(...)
└─ Library::addResource(resource, message)
├── 条码号再次判重（防止并发场景或漏判）
└── resources_.add(resource) 所有权转移给容器
```

校验分三层：界面层拦截格式错误（非数字、超范围），模型层拦截语义错误（校验位不符、条码号重复），业务层再做一次重复检查。重复检查看似冗余，但它使 `addResource` 成为可独立调用的安全接口，不依赖调用方是否已检查。

6.5 多态输出流程

```
Library::searchResources("C++", std::cout)
├── forEach 遍历容器，按条码号/书名/作者/书号匹配 → 收集基类指针到 hits
├── std::sort 按 operator< 以条码号排序
└─ for each ptr: std::cout << *ptr
└─ operator<<(ostream&, const BookResource&)
└── resource.print(os) ← 虚函数，按实际类型分派
├── Book::print 显示 ISBN、页数、分类、30 天、0.20 元
```

```
├── Magazine::print 显示 ISSN、期号、周期、7 天、0.50 元
└── EResource::print 显示授权份数、地址、大小、3 天、0.10 元
```

调用点只有一处（`std::cout << *ptr`），输出形态却有三种。若后续增加“学位论文”类型，只需新增派生类并实现 `print`，本流程一行不改。

6.6 退出与保存

```
用户输入 0 → run() 返回
└── main 继续执行
└─ library.saveAll()
├── resources.txt：逐对象 toRecord()（虚函数产生不同类型记录）
├── readers.txt：逐对象 toRecord()
└── loans.txt：遍历每个借阅人的 loans_，逐条 toRecord(readerId)
└── 打印保存结果，返回退出码
```

若用户以重定向方式输入（输入流提前结束），`Console::readLine` 抛出 `ExitSignal`，`main` 捕获后仍执行同一段保存逻辑，因此数据不会因非正常退出而丢失。

7 语法与机制专题

本章把散落在各文件中的语法点按主题归类，便于对照复习与答辩陈述。

7.1 抽象类、纯虚函数与虚析构

语法	出现位置	解决的问题
纯虚函数 `= 0`	`BookResource`、`Borrower` 的 5 个接口	强制派生类实现，使基类成为“接口契约”
抽象类	同上两个类	禁止实例化，编译期阻断“未指定类型的资源”这种无意义对象
`virtual` 析构	`~BookResource`、`~Borrower`	通过基类指针 `delete` 时正确析构派生类部分
`override`	所有派生类的重写函数	签名写错时编译报错，避免“假重写”
基类指针/引用调用	`searchResources`、`borrowResource`	同一调用在不同实际类型上产生不同行为

判断一个类是否该有虚析构的实用准则：只要该类可能被基类指针 `delete`，就必须有虚析构。本项目容器统一保存 `BookResource*` 并在 `removeAt` 中 `delete`，因此虚析构不是可选项而是必需项。

7.2 类模板与类型参数

```
template <typename T>
class DataStore { /* 内部为 std::vector<T*> items_; */ };

DataStore<BookResource> resources_; // 实例化 1
DataStore<Borrower> readers_; // 实例化 2
```

模板在编译期为每个实参生成独立的类代码，因此 `DataStore<BookResource>` 与 `DataStore<Borrower>` 是两个不相关的类型，互不影响。模板成员函数的定义必须放在头文件中，因为编译器在实例化时需要看到完整定义；这也是 `DataStore.h` 没有对应 `.cpp` 的原因。

7.3 const 正确性

用法	示例	含义

用法	示例	含义
常成员函数	`int getTotal() const`	承诺不修改对象状态，可被常量对象调用
常引用参数	`const Date& today`	只读传参，避免对象拷贝
常量指针参数	`const BookResource* item`	遍历中禁止修改元素
`const` 局部变量	`const int days[12]`	只读表，防止误写

`Library` 的查询类函数全部声明为 `const`，因此可以在 `const Library&` 上调用；而 `addResource`、`borrowBook` 等修改状态的函数不加 `const`。这一区分让编译器帮助检查“查询函数是否意外修改了数据”。

7.4 值传递、引用传递与指针传递

方式	本项目用法	选择理由
值传递	`setPrice(double price)`	基本类型，拷贝成本极低
常引用	`const std::string& title`	字符串拷贝代价高，且无需修改
非 const 引用	`BookResource& resource`、`std::string& message`	需要在函数内修改调用方对象
指针	`BookResource* resource`	需要表达“可能为空”
基类指针	`DataStore<BookResource>` 的元素	多态容器，保存异类对象

借还函数是引用传递的核心示例：传值会导致修改丢失，传指针需要额外判空，传引用既保证修改生效又避免空指针检查。

7.5 拷贝控制与资源所有权

类	拷贝构造	拷贝赋值	原因
`Date`、`LoanRecord`	显式实现	显式实现	无堆资源，逐成员复制即可
`BookResource`	显式实现	未定义	无堆资源，复制语义清晰
`DataStore<T>`	`= delete`	`= delete`	持有对象所有权，复制将导致重复释放

类	拷贝构造	拷贝赋值	原因
`Borrower`	`= delete`	`= delete`	含 `vector` 成员且作为多态基类，禁止切片复制
`Library`	`= delete`	`= delete`	组合两个容器，复制将共享同一批对象

`= delete` 的写法把错误从运行期崩溃前移到编译期报错，这是 C++11 之后表达“不可复制”意图的标准手段。

7.6 文件流与字符串流

类型	用途	位置
`std::ifstream`	只读打开数据文件，逐行读取	`Library::loadAll`
`std::ofstream`	覆盖写出数据文件	`Library::saveAll`
`std::istringstream`	把字符串当输入流解析（切分、转数字）	`Text::split`、`Text::toInt`、`operator>>`
`std::ostringstream`	把多个值格式化为一个字符串	`toRecord`、`toString`、消息拼接

统一使用“流”这一抽象，带来两个好处：同一套 `<<` 运算符既能输出到屏幕也能输出到文件；字符串与数字之间的转换不必使用 `sprintf` / `atoi` 等 C 风格函数，类型安全且不受缓冲区长度限制。

7.7 运算符重载

运算符	定义位置	语义
`operator<<`	`BookResource`（友元）	输出对象，内部调用虚函数 `print`
`operator<<`	`Borrower`（友元）	输出借阅人信息
`operator<<` / `operator>>`	`Date`（友元）	日期的流式读写，支持 `2026-10-29` 格式
`operator<`	`BookResource`	按条码号排序，供 `std::sort` 使用
`operator=`	`Date`	赋值时逐成员复制

`operator<<` 定义为友元而非成员函数，原因是左操作数是 `std::ostream` 而非本类对象，

只能以非成员形式定义；声明为 `friend` 是为了让它能访问私有成员（实际实现中仅调用公有接口，写成普通函数亦可）。

7.8 异常与提前退出

```
class Console {
public:
class ExitSignal {}; // 空类，仅作信号使用
};
```

`ExitSignal` 不携带任何数据，只承担“控制流跳转”职责。选择异常而非返回值传递的原因是：输入函数位于多层调用之内（`readLine` ← `readInt` ← `doBorrow` ← `run`），若用返回值表示结束，每一层都要判断并逐级返回，侵入性极强。异常让 `main` 一次捕获即可，代价是必须保证捕获后仍有资源释放——本项目依靠栈对象的析构自动完成。

7.9 中文编码链路

中文不乱码依赖三处设置保持一致，缺一即出现乱码：

环节	设置	位置
源文件字节	保存为 UTF-8（无 BOM）	编辑器
编译期字符集	`-finput-charset=UTF-8 -fexec-charset=UTF-8`	`build.bat`、`tasks.json`
运行期控制台	`SetConsoleOutputCP(CP_UTF8)`、`SetConsoleCP(CP_UTF8)`	`Console::init()`

前两项保证可执行文件中字符串字面量的字节是 UTF-8，第三项保证控制台按 UTF-8 解释这些字节。若使用 MSVC 编译，应改用 `/utf-8` 选项，否则编译器会按系统代码页（GBK）解释源文件，导致字面量在编译期就被破坏。

8 数据文件规范

三个文件均为纯文本、UTF-8 编码、以 `|` 分隔字段，首行为 `#` 注释。选择文本格式而非二进制的原因是可读、可用记事本直接修改、便于答辩演示；代价是体积较大且解析较慢，对课程项目的数据量而言可以接受。

8.1 resources.txt

```
# 资源: 类型|条码号|书号|书名|作者|出版社|总数|可借|价格|扩展
BOOK|B001|9787115428028|C++ Primer（第 5 版）|Stanley B. Lippman|电子工业出版社|3|3|128|848;计算机
MAG|M001|1004194X|计算机学报|编辑部|中国科学院|5|5|30|12;月刊
EBOOK|E001|9787115546081|C++标准库（电子版）|Nicolai M. Josuttis|数字资源库|2|2|89|http://lib.swust.edu.cn/ebook/001;25
```

字段	类型	说明
类型	枚举字符串	`BOOK` / `MAG` / `EBOOK`，决定还原为哪个派生类
条码号	字符串	馆内唯一，程序不校验其格式，仅要求非空
书号	字符串	ISBN 或 ISSN，写入前已通过校验位验证
书名、作者、出版社	字符串	写入前经 `sanitize` 处理，不含 `|` 与 `;`
总数、可借	整数	可借数大于总数时读入后自动修正为总数
价格	浮点	两位小数显示，文件中保留原始精度
扩展	字符串	派生类特有字段，以 `;` 分隔；图书为“页数;分类”

8.2 readers.txt

```
# 借阅人: 身份|编号|姓名|院系|最大借阅数|扩展
STUDENT|20250001|张三|计算机科学与技术学院|5|软件工程;计科 2501
TEACHER|T201501|王老师|计算机科学与技术学院|10|副教授
```

8.3 loans.txt

```
# 借阅记录: 编号|条码号|书名|借出日期|应还日期
20250001|B002|数据结构（C++版）|2026-09-19|2026-10-19
```

借阅记录独立成文件而不嵌入 `readers.txt`，原因是二者是“一对多”关系：嵌入会导致借阅人一行过长且难以扩展，独立成行则天然支持任一借阅人有多条记录。

8.4 容错规则

异常情况	处理
文件不存在	视为首次运行，生成演示数据
行以 `#` 开头或为空	跳过
字段数不足 10（资源）/ 6（借阅人）/ 5（记录）	跳过该行
类型标记未知	跳过该行
扩展字段解析失败	`delete` 已创建对象并跳过
条码号或编号重复	丢弃后出现的一条
可借数大于总数或为负	修正为总数或 0

9 编译与运行配置

9.1 编译命令

```
g++ -std=c++17 -Wall -Wextra -g
-finput-charset=UTF-8 -fexec-charset=UTF-8
main.cpp Utils.cpp BookResource.cpp Borrower.cpp Library.cpp ConsoleUI.cpp
-o build\Library.exe
```

参数	作用
`-std=c++17`	启用 C++17，`std::filesystem` 与 `std::function` 依赖此标准
`-Wall -Wextra`	开启常用与额外警告，工程在此设置下为零警告
`-g`	生成调试信息，供 gdb 断点使用
`-finput-charset` / `-fexec-charset`	固定源文件与执行字符集为 UTF-8
`-o build\Library.exe`	指定输出路径

工程未使用 `-I` 参数，因为所有头文件与源文件同目录。

9.2 VSCode 配置

文件	作用	关键字段
`.vscode/tasks.json`	定义 build 与 run 两个任务	`build` 调用 `build.bat`；`run` 以 `dependsOn` 依赖 build
`.vscode/launch.json`	配置 F5 调试	`miDebuggerPath` 指向 `gdb.exe`，`preLaunchTask` 为 build
`.vscode/c_cpp_properties.json`	IntelliSense 解析	`includePath` 为 `${workspaceFolder}`，`cppStandard` 为 `c++17`

`tasks.json` 中的 `presentation.panel` 设为 `dedicated`，使运行输出独占一个终端面板，不与编译输出混排；`build.bat` 采用纯 ASCII 注释与 CRLF 换行，因为 cmd.exe 按系统 OEM 代码页解析批处理文件，中文注释可能被误判为命令。

9.3 数据目录与工作目录

程序的三个数据文件放在工程根的 `data` 目录下。这个目录曾被写成相对路径 `"data"`，它的真实含义是“当前工作目录下的 data”，而工作目录由启动方式决定，于是同一个程序会读写到两份不同的数据：

启动方式	工作目录	旧行为（相对路径）	现行为（`defaultDataDir`）
VSCode 的 `run` 任务	工程根	`data\`	`data\`
F5 调试（`launch.json`）	工程根	`data\`	`data\`
在 `build` 目录执行 `Library.exe`	`build\`	`build\data\`（新建副本）	`data\`
双击 `build\Library.exe`	`build\`	`build\data\`（新建副本）	`data\`

后两种情况下，`build\data\` 里是一份全新的演示数据，用户会以为“刚才借的书丢了”。修复方式即 5.5.7 节的 `defaultDataDir`：以可执行文件位置为基准向上找到带 `build.bat` 的工程根，再拼 `data`。

三个配套约定：

1. **工程根的唯一标志是 `build.bat`**。移动或改名该脚本会让定位回退到“数据放 exe 旁边”，因此不要重命名它。

2. **`build` 目录只放编译产物**，不放数据。`build.bat` 中只 `mkdir build`，不创建任何数据子目录。

3. **临时改数据位置**：`build\Library.exe D://some//where`，第一个命令行参数会被当作数据目录（见 5.7 节语法点五），适合答辩时用干净数据演示而不影响日常数据。

   排错顺序：先看程序退出时打印的“数据已保存到 …… 目录”一行，确认它指向工程根；若指向别处，检查 `build.bat` 是否还在工程根、exe 是否被复制到了其他目录。

10 维护指南

10.1 新增一种资源类型

以“学位论文”为例，需要修改三处，业务层与界面层的公共代码无需改动：

1. `BookResource.h`：新增 `class Thesis : public BookResource`，实现 `typeName`、`typeTag`（如 `"THESIS"`）、`maxBorrowDays`、`finePerDay`、`print`、`extraData`、`loadExtra`。
2. `BookResource.cpp`：实现上述函数，并在 `fromRecord` 的类型分派中增加一个 `else if (fields[0] == "THESIS")` 分支。
3. `ConsoleUI.cpp`：在 `inputNewResource` 中增加一个类型选项与对应字段读取。

`Library`、`DataStore`、`Borrower` 均不需要修改，多态容器与虚函数调用会自动容纳新类型。这是本设计最主要的可扩展性收益。

10.2 调整借阅或罚款规则

需求	修改位置
改变某类资源的借期	对应派生类的 `maxBorrowDays()` 返回值
改变日罚金	对应派生类的 `finePerDay()` 返回值
改变借阅上限	对应派生类构造函数的第四个实参
改变折扣政策	对应派生类的 `fineDiscount()` 返回值
改变罚款公式	`Borrower::returnResource` 中的一行乘法

上述修改均不涉及数据结构与文件格式，改动后无需迁移已有数据。

10.3 扩展数据字段

在记录末尾追加字段是兼容性最好的扩展方式：`toRecord` 末尾增加一个字段，`loadExtra` 中按 `;` 多解析一段，旧数据文件在该字段缺失时由 `Text::toInt` 的默认值兜底，无需手工迁移。若在中间插入字段，则必须同步修改 `fromRecord` 的下标并处理旧文件。

10.4 已知限制

1. 数据全部驻留内存，适合万级以下记录；更大规模应改为按需查询的数据库方案。
2. 借阅历史在归还后即被删除，不保留完整流水；如需统计借阅频次，应增加独立的日志文件。
3. 无并发控制与用户认证，单机单用户使用。

4. 罚款仅提示不扣款，无支付流程。
5. 清屏依赖 `system("cls")`，仅 Windows 有效；跨平台需替换为 ANSI 转义序列。
6. 界面按精简要求只保留 11 项功能，修改资源、修改借阅人、增减馆藏数量、显示全部列表等入口已移除；对应的模型层接口（`setTitle`、`setMaxBooks`、`changeTotal`）仍然保留，需要恢复时只需在 `ConsoleUI` 中新增一个分支函数即可，业务层与模型层无需改动。

11 附录

11.1 函数调用速查

用户操作	界面函数	业务函数	模型函数
关键字查询	doSearchResource	searchResources	各派生类 print、getTitle
添加资源	doAddResource	addResource	构造函数、isValidStdNumber
删除资源	doRemoveResource	removeResource	getBorrowed
添加借阅人	doAddReader	addReader	isValidId
借书	doBorrow	borrowBook	Borrower::borrowResource
还书	doReturn	returnBook	Borrower::returnResource
在借记录	run 分支 8	listLoans(false)	LoanRecord::toString
逾期清单	run 分支 9	listLoans(true)	LoanRecord::overdueDays
统计	run 分支 10	statistics	getTotal、roleName
退出保存	run 返回	saveAll	toRecord、extraData

11.2 常见问题

现象	原因	处理
中文显示为乱码	源文件编码、编译选项、控制台代码页三者不一致	确认 UTF-8 保存、编译带字符集选项、Console::init 被调用
编译报 filesystem 未定义	未启用 C++17	添加 -std=c++17
启动即退出并提示保存失败	程序目录不可写	将工程放到有写权限的目录
F5 无法启动调试	未安装 gdb	执行 pacman -S mingw-w64-x86_64-gdb
数据被清空后重启又出现演示数据	删除的是文件内容而非文件	保留 resources.txt 文件即可（文件存在即不重建）
换了启动方式后“数据不见	相对路径基准是工作目	已由

现象	原因	处理
了”	录，曾在 `build\` 下另建一份 `data`	`Library::defaultDataDir` 统一；确认退出提示里的路径指向工程根

11.3 术语对照

中文	英文/关键字	本项目对应
抽象类	abstract class	`BookResource`、`Borrower`
纯虚函数	pure virtual function	`print`、`typeName`、`maxBorrowDays`
虚析构函数	virtual destructor	`~BookResource`、`~Borrower`
类模板	class template	`DataStore<T>`
组合	composition	`Library` 与容器、`Borrower` 与 `LoanRecord`
依赖	dependency	`Borrower::borrowResource(BookResource&)`
多态	polymorphism	基类指针调用 `print`
序列化	serialization	`toRecord` / `fromRecord`

结语

本项目的实现重点不在于功能数量，而在于用一条清晰的类型体系承载业务规则：`BookResource` 与 `Borrower` 两条继承链把“借什么”与“谁来借”的差异收敛到少数虚函数中，`Library` 只负责组织与协调，`ConsoleUI` 只负责交互。理解这一分工之后，读者定位任何一个功能都只需沿着“界面 → 业务 → 模型”三层向下追踪，这也是本说明书按此顺序组织章节的原因。